

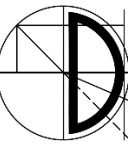
Grundlagen der Programmierung

Kapitel 1: Grundbegriffe

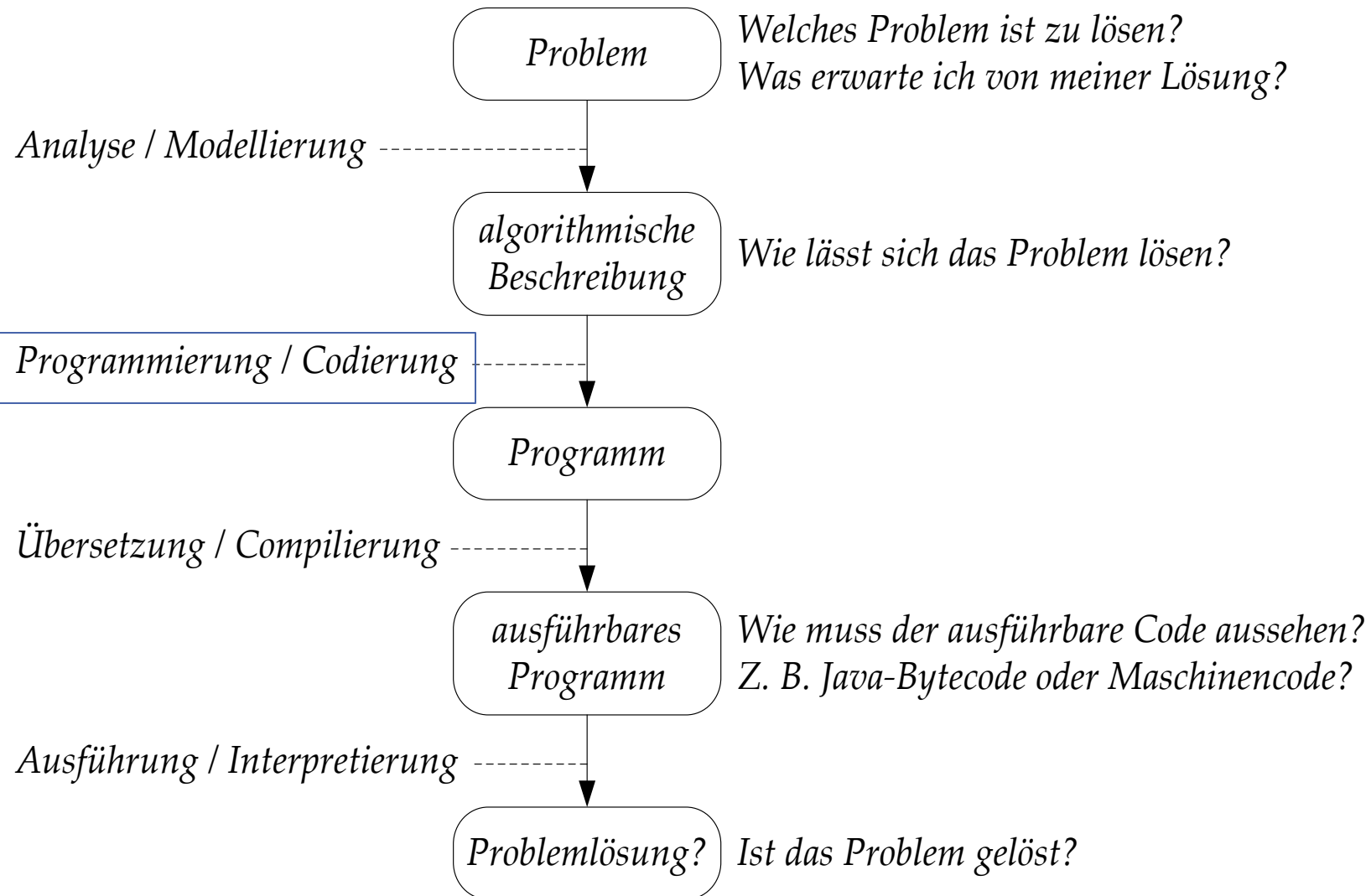
Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

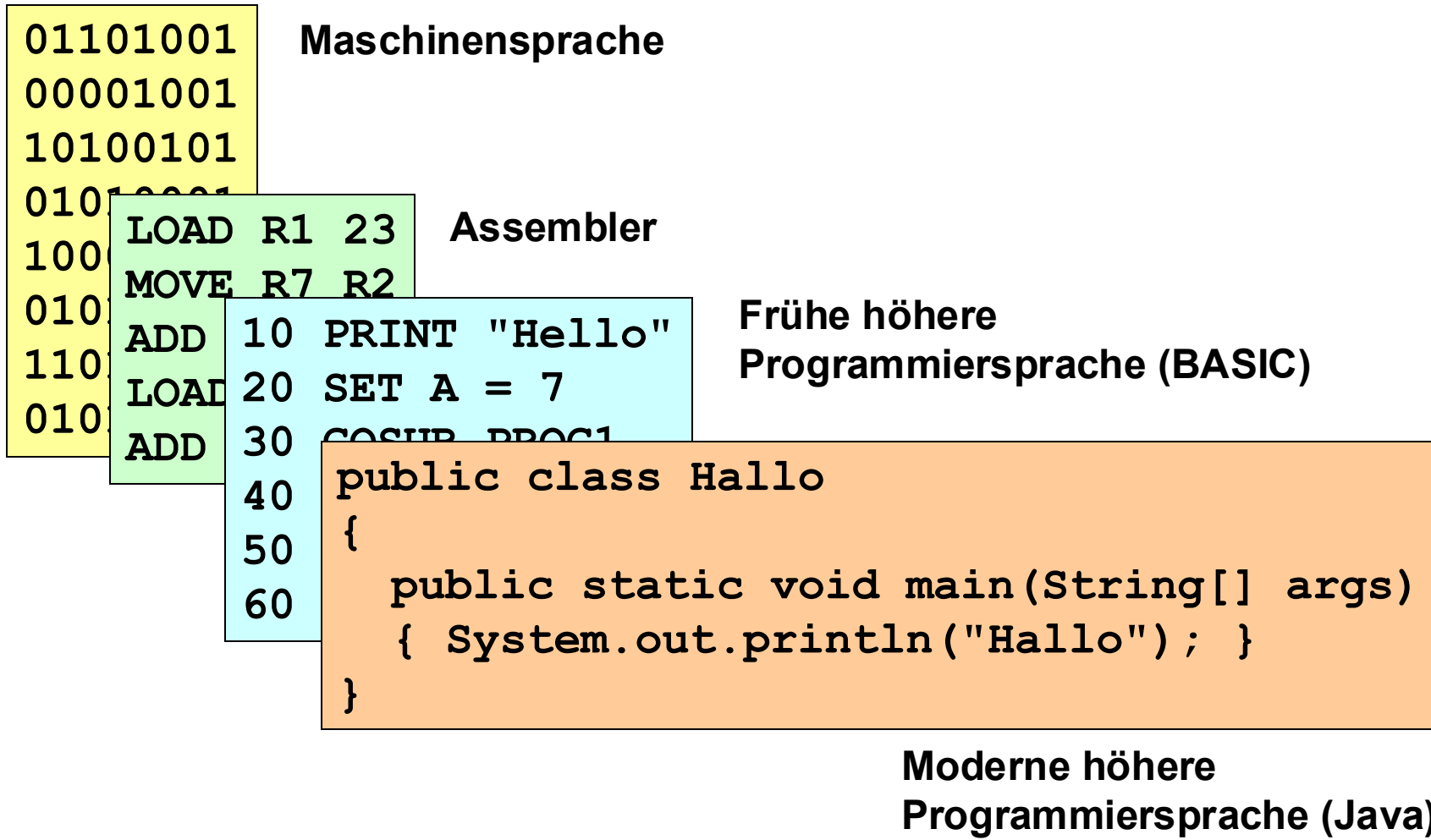
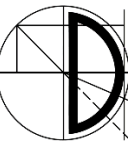
- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

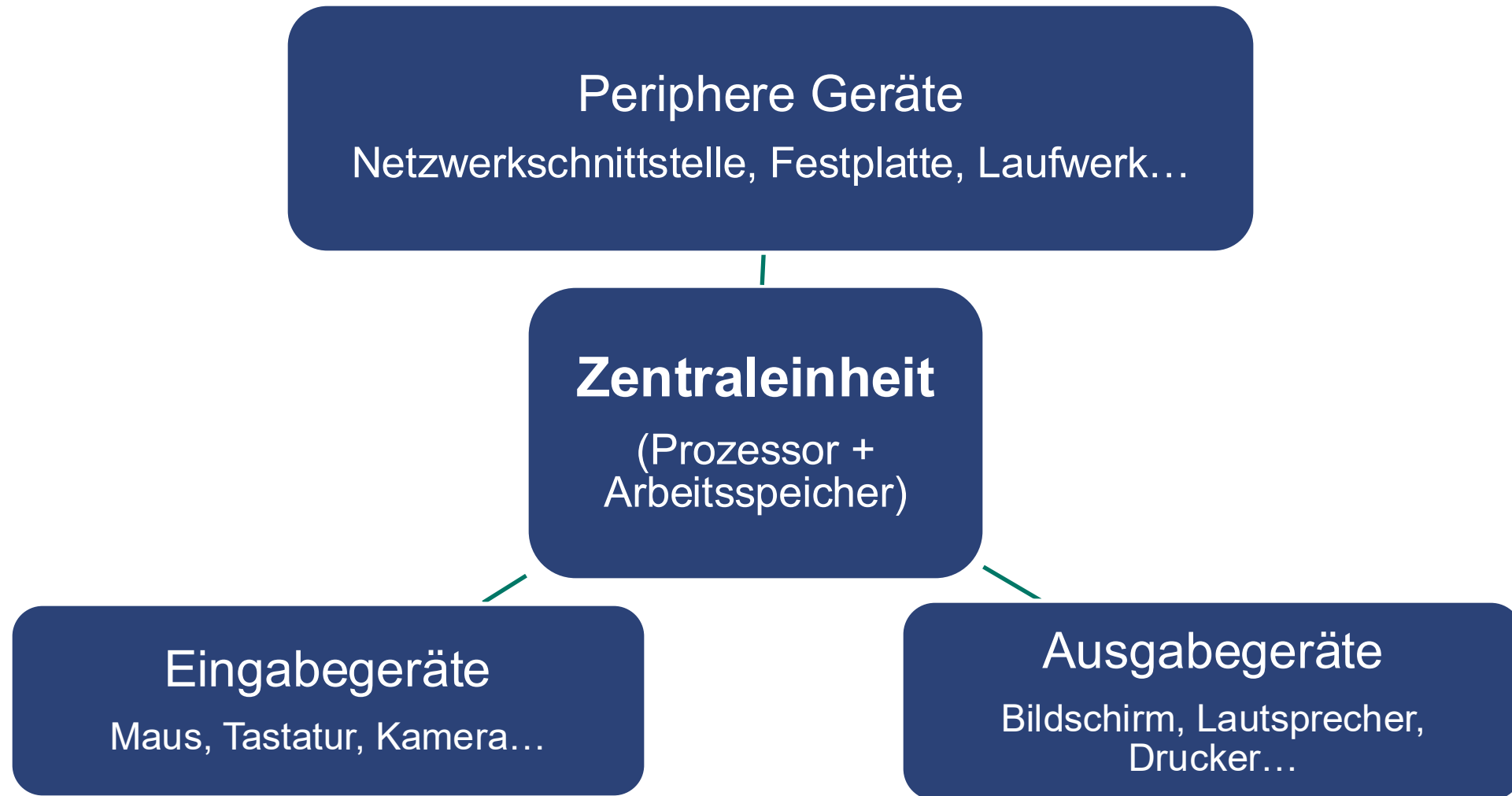
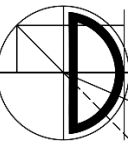
- Was heißt Programmieren?
- Aufbau von Rechenanlagen: Hardware und Software
- Compiler, Interpreter, Zwischensprachen
- Grundbegriffe der Programmierung

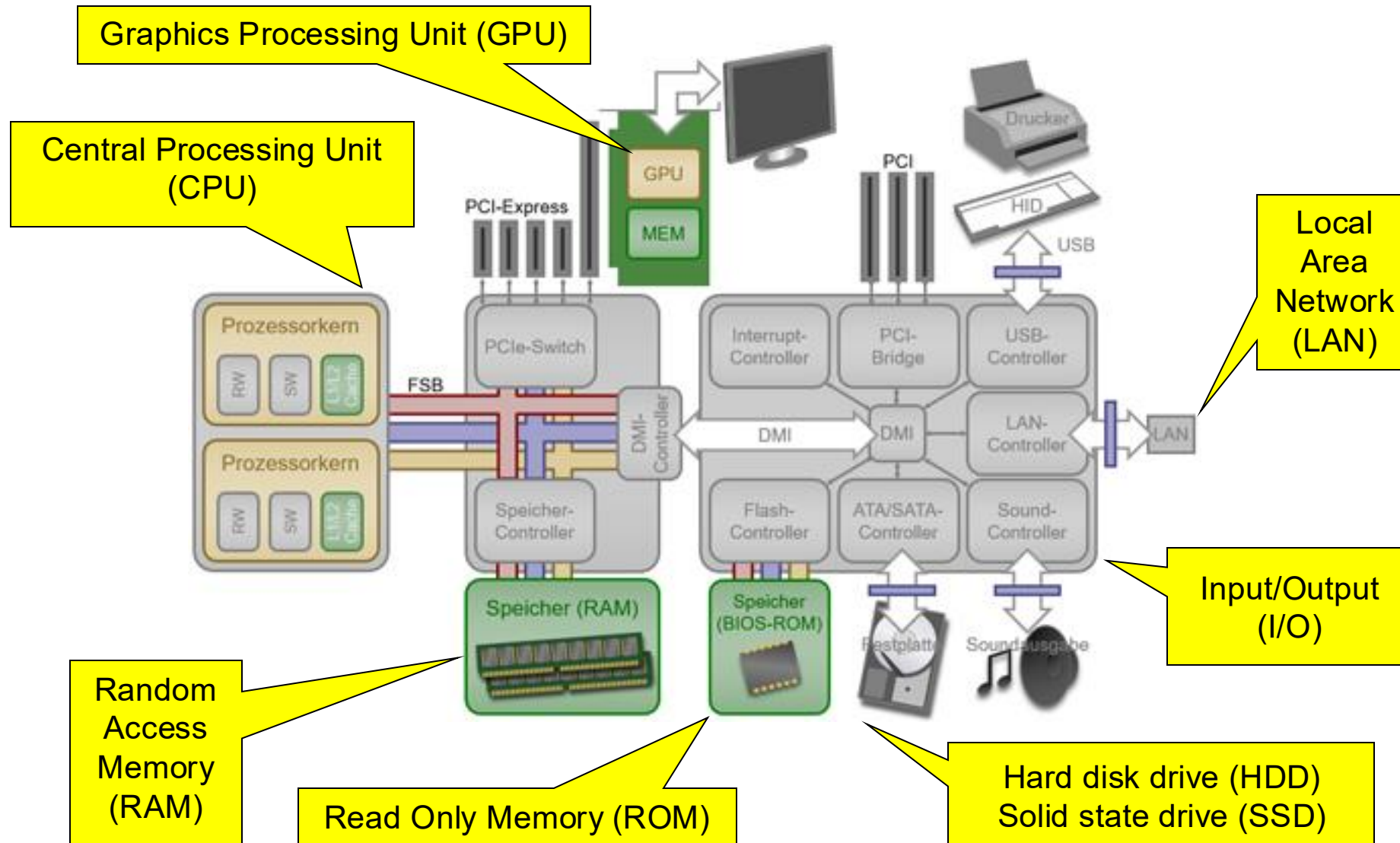


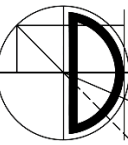
Umsetzen eines
gegebenen
Algorithmus in
ein lauffähiges
Computer-
Programm



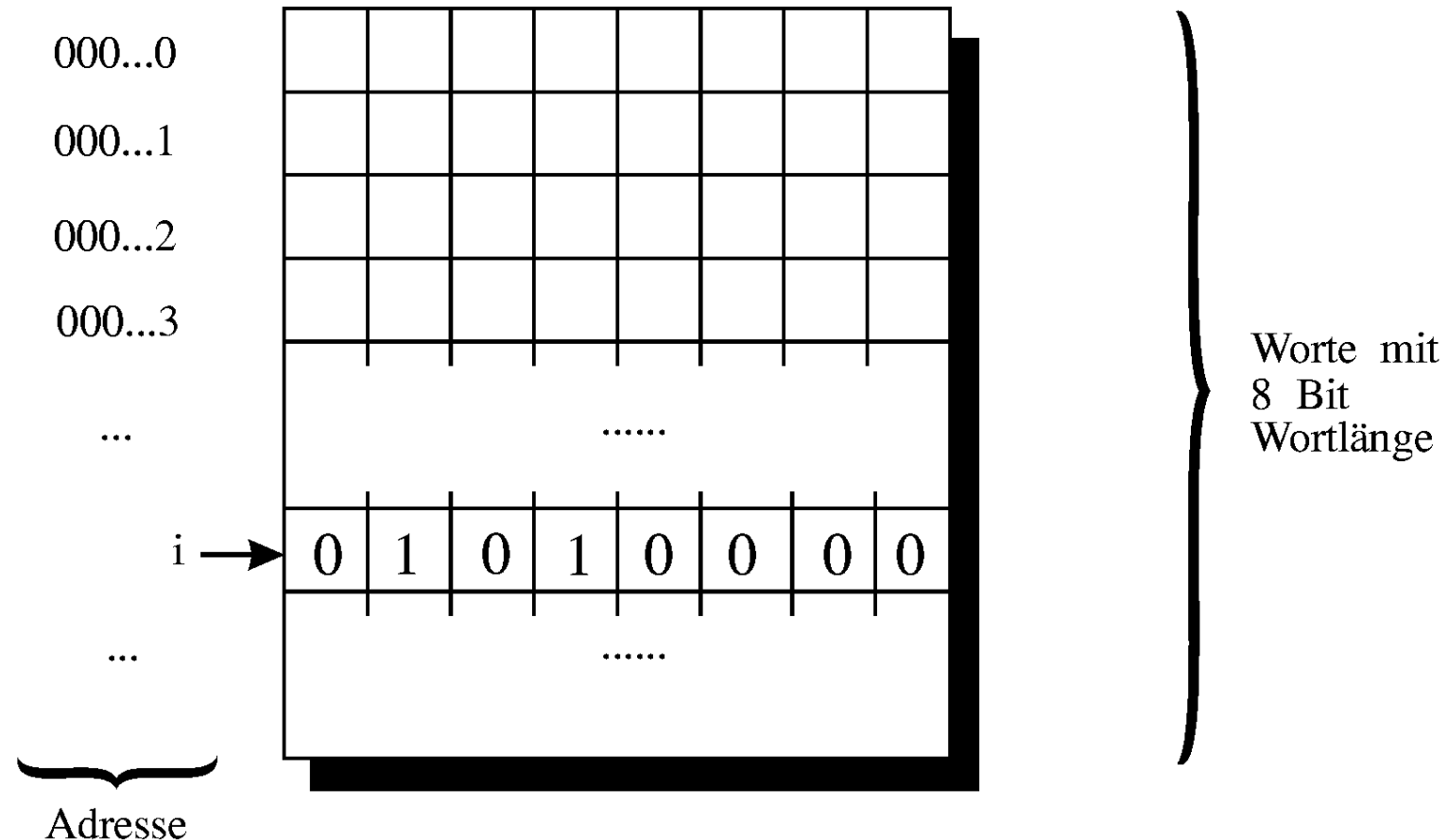






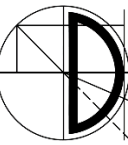


- Einfaches Schema des Arbeitsspeichers/Hauptspeichers (main memory, RAM)



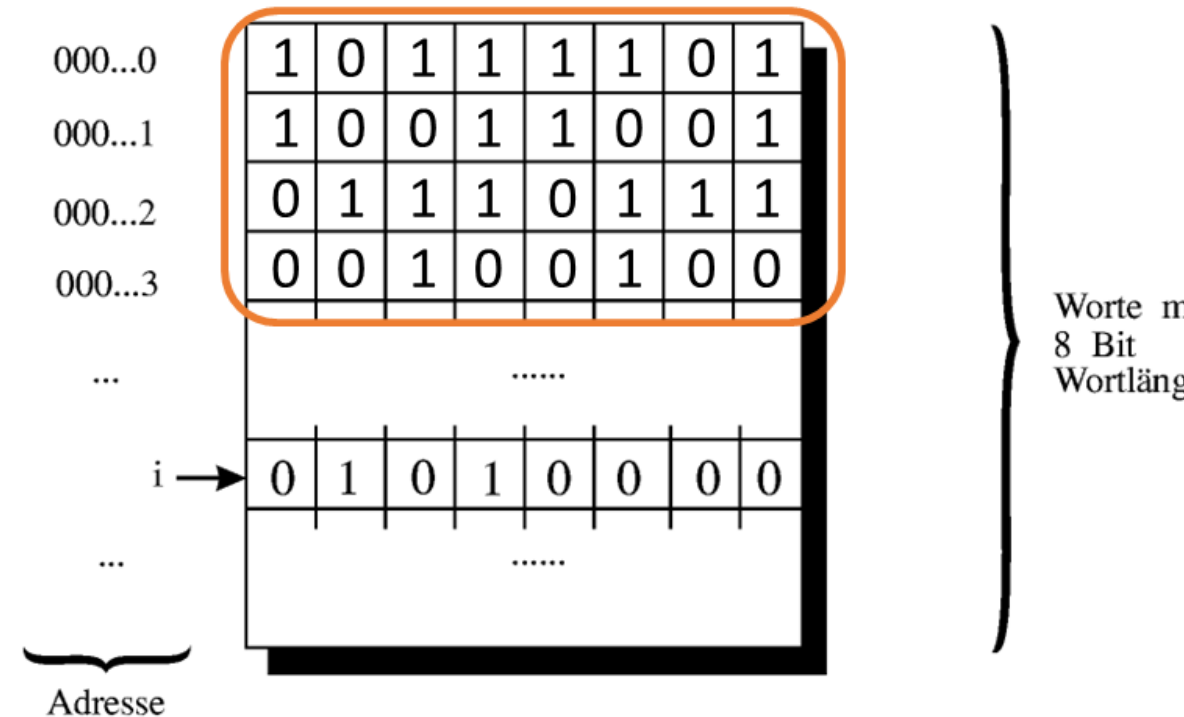
Worte mit
8 Bit
Wortlänge

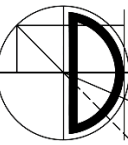
Adresse



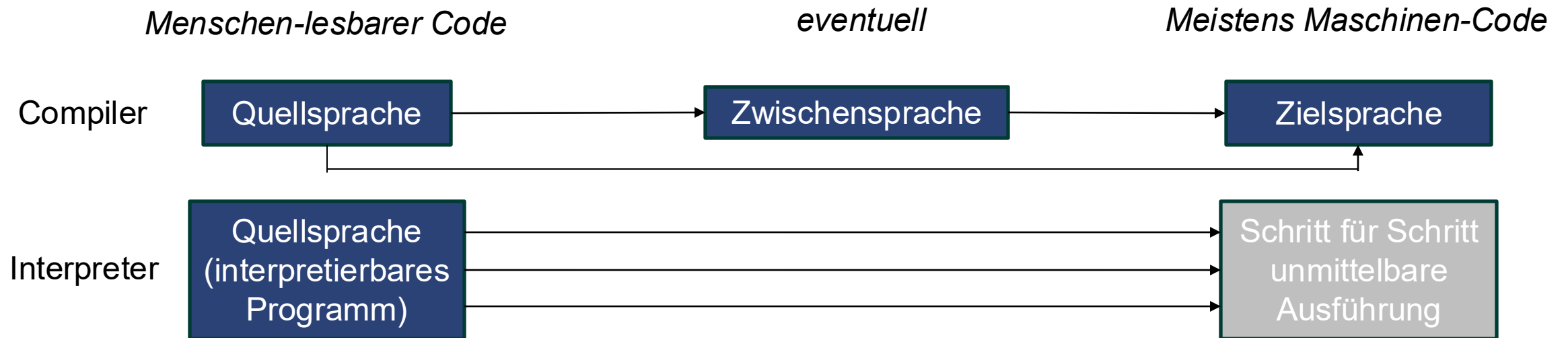
- Datenspeicherung im Arbeitsspeicher
 - einzelne Daten können sich (abhängig vom Typ) aus mehreren Speicherworten zusammensetzen
- Verbindung zu Programmiersprachen
 - Adressen im Speicher erhalten symbolische Namen (z.B. Variablen)
 - Inhalte mehrerer Speicherworte werden kombiniert und ergeben bestimmte Werte (z.B. Werte von Variablen)

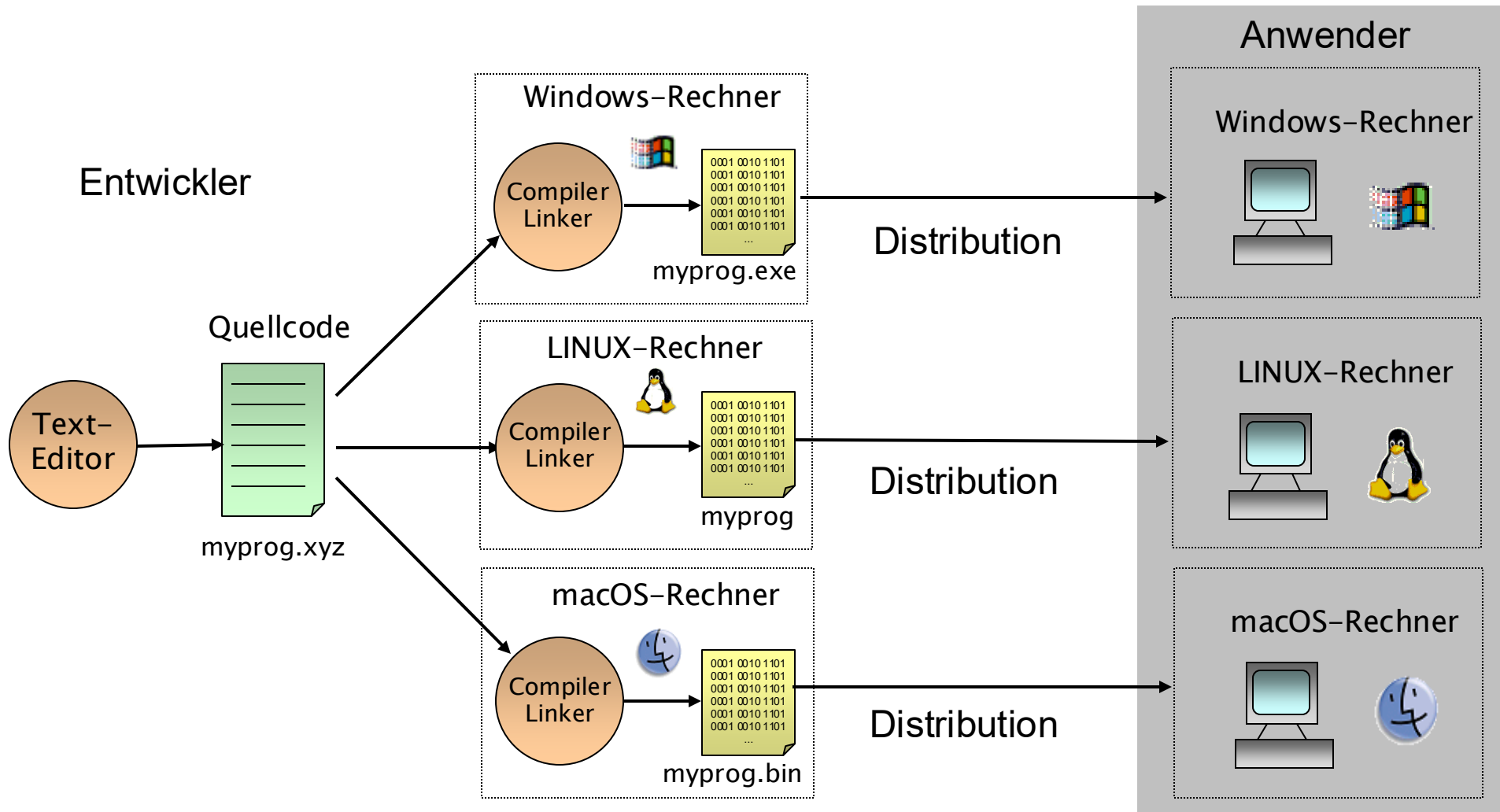
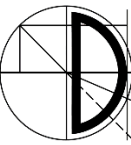
Diese Worte zusammen könnten einen 32-bit Integer (Ganzzahl) darstellen, im Programm könnte dieser den Namen *x* haben

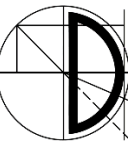




- Klassifizierung nach Norm ISO/IEC 2382:
 - **Systemsoftware** (z.B. Betriebssystem, Treiber) → ermöglicht das Ausführen von Anwendungen
 - **Unterstützungssoftware** (z.B. Texteditor, Compiler, Interpreter) → erbringen eine nicht-anwendungsspezifische Leistung
 - **Anwendungssoftware** (z.B. Office-Produkte, Spiele) → bieten direkten Nutzen für den Endnutzer
- Compiler und Interpreter
 - Werkzeuge, die lesbaren Programm-Quelltext (source code) verarbeiten





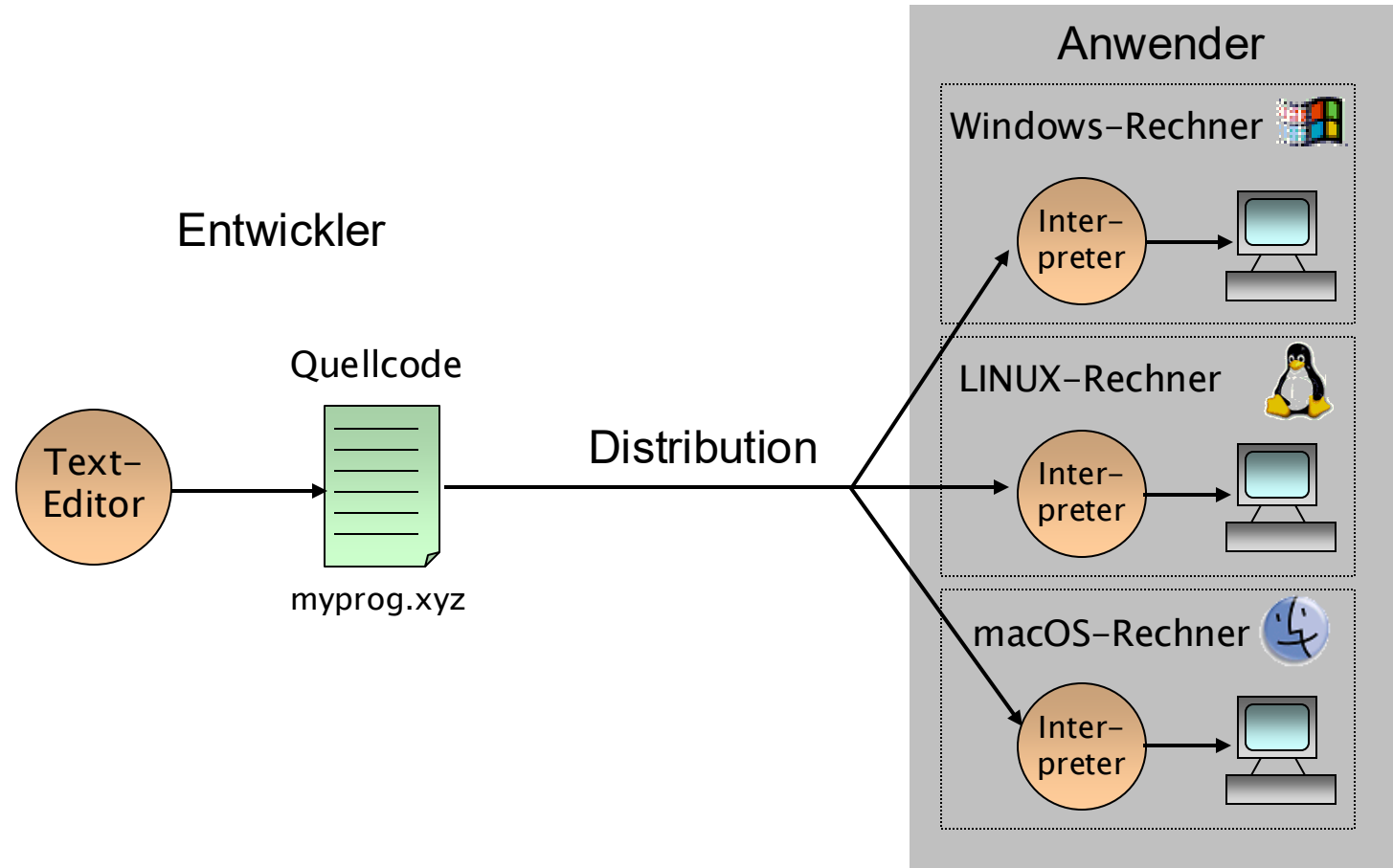
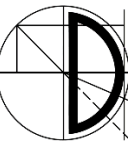


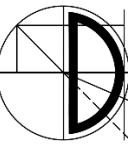
■ Vorteile

- Optimale Nutzung der Prozesseureigenschaften
- Hohe Abarbeitungsgeschwindigkeit
- Quellcode ist geschützt

■ Nachteile

- Programme sind spezialisiert auf Zielrechner
- Zusätzlicher Aufwand für Anpassungen (z.B. grafische Oberflächen)
- Verschiedene Compiler notwendig





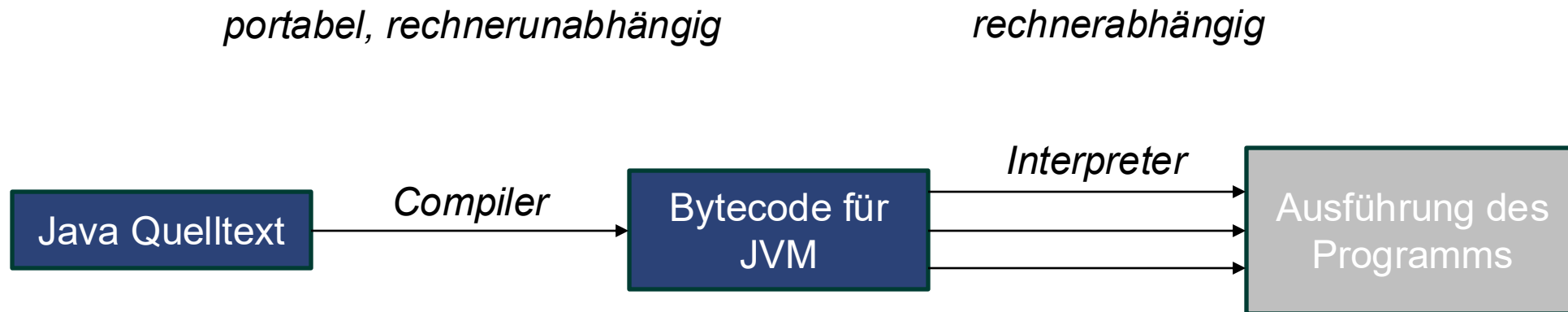
■ Vorteile

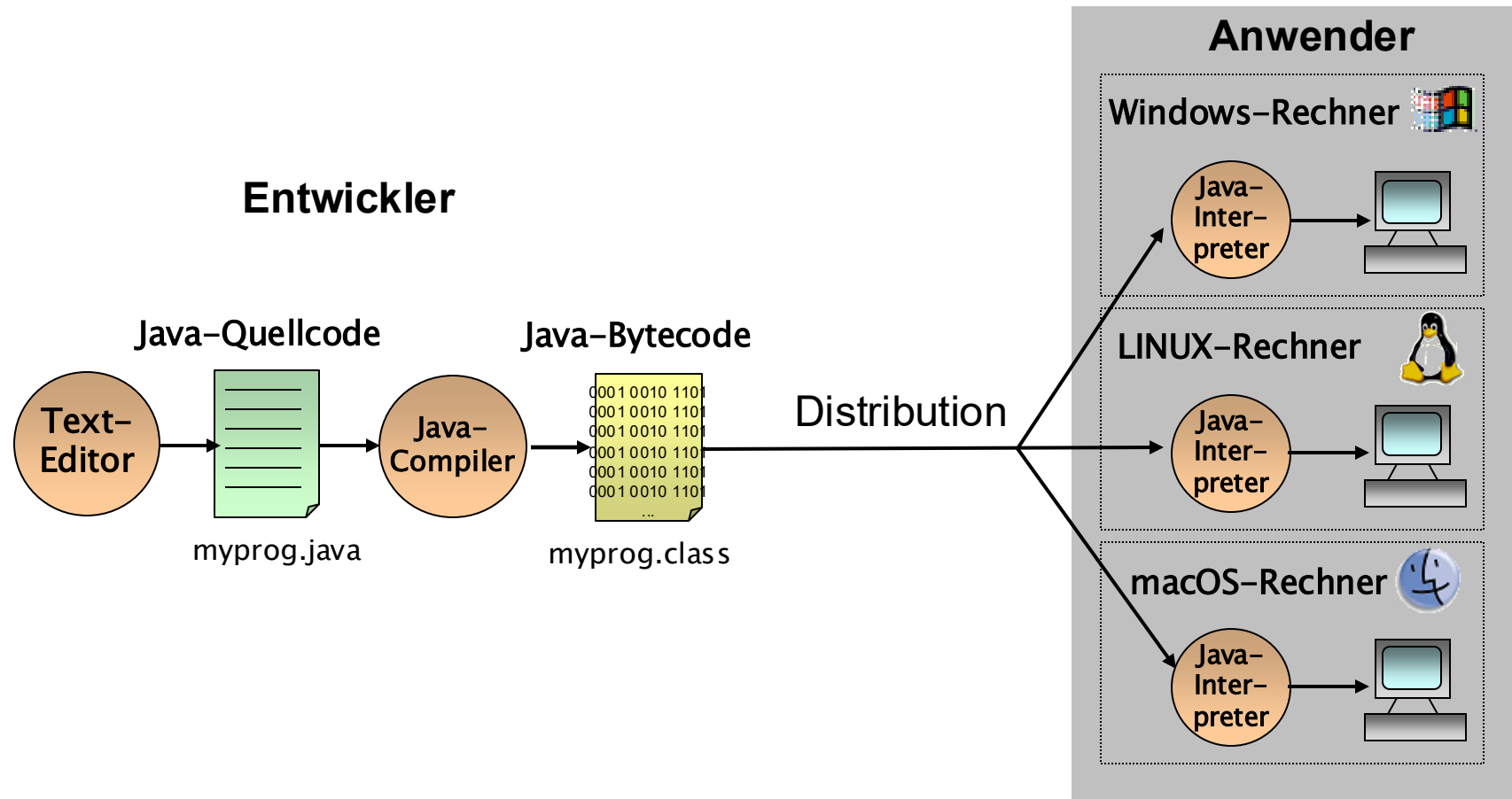
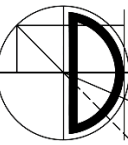
- Quellcode direkt ausführbar, sofern Interpreter für Zielrechner verfügbar

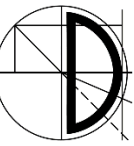
■ Nachteile

- Langsame Ausführung
- Verschiedene Interpreter notwendig
- Quellcode lesbar

- Kombination von Compiler und Interpreter für Code einer virtuellen Maschine (**Java Virtual Machine – JVM**)





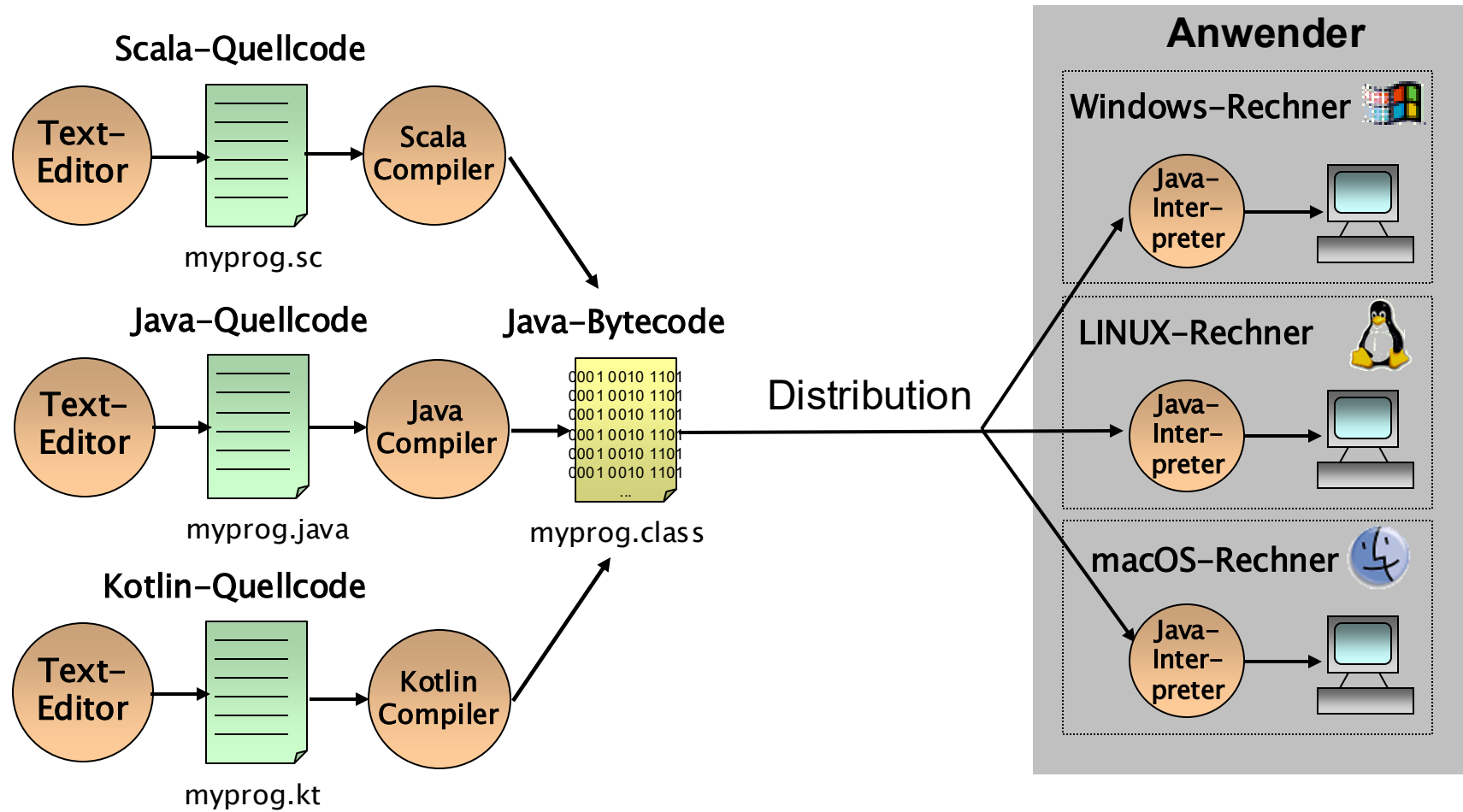
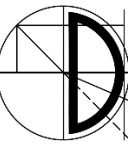


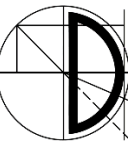
■ Vorteile

- Nur ein Compiler
- Sprache ist plattformunabhängig
- Quellcode teilweise geschützt

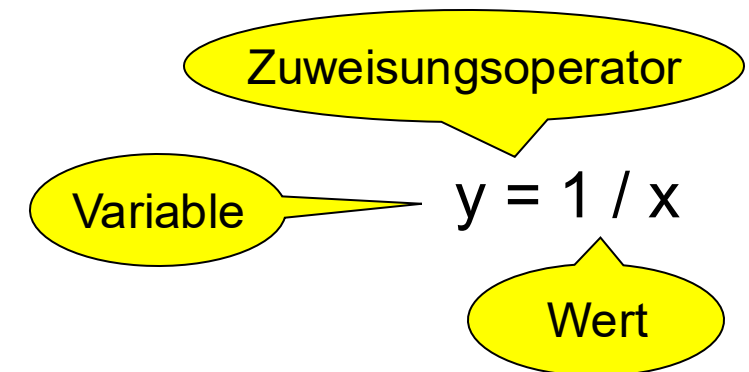
■ Nachteile

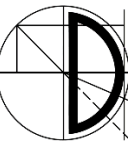
- Langsamer als Maschinencode
- Verschiedene Interpreter notwendig





- **Variable** (**variable**)
 - "Platzhalter" für einen Wert eines bestimmten Typs
 - Ist mit einer Adresse im Hauptspeicher assoziiert
 - Beispiel:
 - Algorithmus A: "Kehrwert von 4"
 - dividiere 1 durch 4
 - Algorithmus B: "Kehrwert einer Zahl"
 - setze x auf den Wert der Zahl
 - dividiere 1 durch x
- **Zuweisung (Wertzuweisung)** (**assignment**)
 - Weist einer Variable einen Wert zu
 - Beispiel:
 - $x = 5$

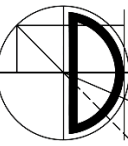




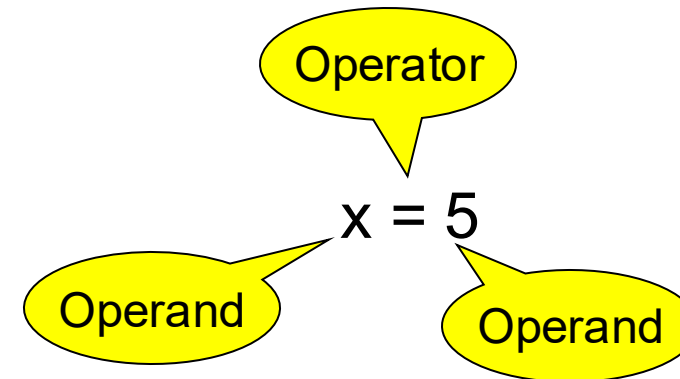
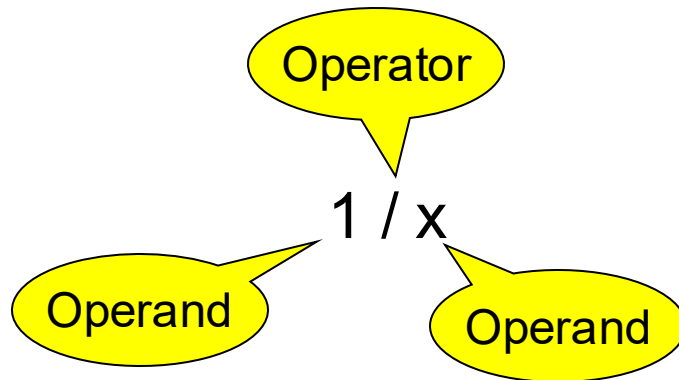
■ Datentyp (Typ) (data type)

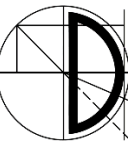
- Bauplan für Daten (Werte von Variablen oder Konstanten), der festlegt:
 - Wie die **Darstellung** der Werte im Speicher erfolgt,
 - Welche **Operationen** für die Werte erlaubt sind,
 - Welche **Standardwerte** (default values) festgelegt sind
- Beispiele für Variablen im Speicher (konzeptionell, ungenau):

Variablenname	Speicheradresse	Speicherinhalt	Datentyp
x	001...0100	0 1 0 0 0 1 0 1 0 0 0 1 0 1 0 0	ganze Zahl (z.B. in Java: <i>short</i> , 16 bit)
y	010...1001	0 1 0 1 0 1 0 1 1 1 0 0 0 1 0 1 1 1 1 0 0 1 0 1 0 1 0 0 0 1 0 1	Gleitkommazahl (z.B. in Java: <i>float</i> , 32 bit)



- **Ausdruck** (*expression*)
 - Kombination von Operanden und Operatoren als „Vorschrift“ zur Berechnung eines Wertes
 - Liefert immer einen Wert (Ergebniswert) ab
 - Auch Zuweisungen werden als Ausdrücke betrachtet





■ Anweisung (**statement**)

- Kombination von Ausdrücken und Methoden als „Vorschrift“ zur Ausführung einer Aktion
- Wird abgeschlossen durch ein Satzzeichen
 - ; im Fall von Java und C/C++

■ Beispiele:

`x = 5;`

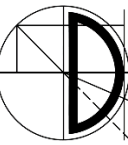
Wertzuweisung

`y = 1 / (x + 1);`

Wertzuweisung

`print(x);`

Ausgabeanweisung (Methodenaufruf)



- **Deklaration** (declaration)

- Festlegen des Namens und Datentyps einer Variablen

- Beispiele:

- Deklaration:

- `int a;`
- `char b;`

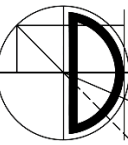
- Zuweisung:

- `a = 2;`
- `b = '#';`

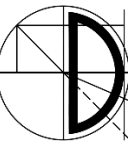
- Deklaration und Zuweisung (Initialisierung):

- `int c = a + 2;`

Java**C/C++**



- **Anweisungssequenz** (sequence of statements)
 - Mehrere Anweisungen, die nacheinander ausgeführt werden
- **Anweisungsblock** (block)
 - Logisch zusammengefasste Anweisungen bzw. Programmteile, die als eine Anweisung aufgefasst werden können
 - Sind begrenzt durch spezielle Zeichen (in Java, C/C++: durch { und })
- **Bedingte Anweisung** (conditional statement)
 - Anweisung oder Anweisungsblock, dessen Ausführung an eine Bedingung gebunden ist
- **Wiederholungsanweisung/Schleife** (loop)
 - Mehrfach ausgeführte Anweisung oder Anweisungsblock



- Schleife mit Block

```
int a = 1;  
while (a < 256) {  
    a = a * 2;  
}
```

Java

C/C++

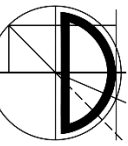
- Bedingte Anweisung

```
if (a < 5) {  
    a = a * 3;  
} else {  
    a = a / 4;  
}
```

Java

C/C++

One last thing...



Schon gewusst, dass die Programmiersprache R Zuweisungen auch „von links nach rechts“ erlaubt?

R ist eine Programmiersprache, die z.B. für statistische Analysen genutzt wird. Vorgänger dieser Sprache konnten nur auf der Konsole ausgeführt werden, wo man nicht einfach an den Beginn einer Zeile springen konnte. Wenn man eine lange Formel eingetippt hatte, erlaubte es der rechte Pfeiloperator das Ergebnis doch noch in eine Variable zu speichern, falls man es am Anfang der Zeile vergessen hatte.

Zuweisungen in R:

`x <- 2` oder `2 -> x`



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'U' and 'W' shape. The text 'Julius-Maximilians-' is in a smaller font above the larger, bold text 'UNIVERSITÄT WÜRZBURG'.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit